

RELACIONES ENTRE CLASES

Asociación · Agregación · Composición · Herencia

RELACIÓN	TIPO	FRASE CLAVE	VIDA INDEP.
Asociación	Débil	"usa / trabaja con"	✓ Sí
Agregación	Media	"tiene un" (externo)	✓ Sí
Composición	Fuerte	"es parte de"	✗ No
Herencia	Muy fuerte	"es un" (ES-UN)	✗ No

🔗 ASOCIACIÓN – "TRABAJA CON"

```
class Empleado {
    private Empresa empresa; // referencia externa
    public Empleado(String n, Empresa e) {
        this.empresa = e; // se PASA, no se crea
    }
}
```

🔗 AGREGACIÓN – "TIENE UN" (INDEPENDIENTE)

```
class Coche {
    private Motor motor; // Motor creado FUERA
    public Coche(String marca, Motor motor) {
        this.motor = motor;
    }
}
Motor m = new Motor(150); // vive solo
Coche c = new Coche("Toyota", m);
```

🔗 COMPOSICIÓN – "ES PARTE DE" (DEPENDIENTE)

```
class Persona {
    private class Corazon { // clase interna privada
        public void latir() { /*...*/ }
    }
    private Corazon corazon;
    public Persona(String n) {
        corazon = new Corazon(); // se crea DENTRO
    }
}
```

✓ Agregación
Objeto se *pasa* por parámetro.
La parte puede vivir sola.

🔗 Composición
Objeto se *crea dentro*.
La parte *no* existe sola.

🚀 REGLA RÁPIDA

¿Solo se conocen? → Asociación

¿Una tiene la otra (fuera)? → Agregación

¿Una la crea dentro? → Composición

¿Una ES la otra? → Herencia

HERENCIA · CASTING · POLIMORFISMO

extends · super · @Override · instanceof

🚀 HERENCIA – EXTENDS

```
class Animal {
    protected String nombre;
    public Animal(String n) { this.nombre = n; }
    public void hacerSonido() { /*...*/ }
}
class Perro extends Animal {
    public Perro(String n) {
        super(n); // ← 1ª línea siempre
    }
    @Override
    public void hacerSonido() {
        System.out.println("Guau");
    }
}
```

MODIFICADOR	¿SE HEREDA?	¿ACCESIBLE EN HIJA?
private	✗ No	✗ No (solo getters)
protected	✓ Sí	✓ Sí directo
public	✓ Sí	✓ Sí
Constructores	✗ NO	Solo vía <code>super()</code>

3 USOS DE SUPER

`super(args)` Llama al constructor padre — 1ª línea

`super.atrib` Accede a atributo del padre (oculto)

`super.metodo()` Llama al método padre antes del propio

🔄 CASTING

```
// UPCASTING – automático, siempre seguro
Animal a = new Perro("Firu"); // ← Hija→Padre ✓

// DOWNCASTING – explícito, puede fallar
if (a instanceof Perro p) { // Java 16+ pattern
    p.ladrrar(); // seguro ✓
}
// Forma clásica
if (a instanceof Perro) {
    Perro p = (Perro) a; // cast explícito
}
// ERROR típico:
Perro p = (Perro) new Animal(); // ClassCastException!
```

🐾 POLIMORFISMO

Se ejecuta el método de la **clase real** del objeto, no el de la referencia.

```
Animal[] animales = {
    new Perro("Firu"), new Gato("Misi")
};
for (Animal a : animales) {
    a.hacerSonido(); // Guau / Miau
    if (a instanceof Perro p) p.ladrrar();
}
```

CLASES ABSTRACTAS · FINAL

abstract · final · contrato parcial

🏗️ CLASE ABSTRACTA – ABSTRACT CLASS

Idea: plantilla parcial, no instanciable. Define esqueleto que las subclases deben completar.

```
abstract class Animal {
    protected String nombre;
    public Animal(String n) { this.nombre = n; }

    // sin cuerpo → obliga a subclase
    public abstract void hacerSonido();

    // método concreto compartido
    public void dormir() {
        System.out.println(nombre + " duerme");
    }
}
class Perro extends Animal {
    public Perro(String n) { super(n); }
    @Override
    public void hacerSonido() {
        System.out.println("Guau");
    }
}
```

```
// Animal a = new Animal(); ← ERROR
Animal a = new Perro("Firu"); // ✓ polimorfismo
a.hacerSonido(); // Guau
a.dormir(); // Firu duerme
```

PUUEDE TENER	¿SÍ/NO?
Métodos abstractos (sin cuerpo)	✓ Sí
Métodos concretos (con cuerpo)	✓ Sí
Atributos de instancia	✓ Sí
Constructor	✓ Sí (vía super)
Instanciarse directamente	✗ No

🔒 MODIFICADOR FINAL – 3 USOS

🔒 MÉTODO FINAL – NO SE SOBRESCRIBE

```
class CuentaBancaria {
    public final void mostrarSaldo() { /*...*/ }
}
class CuentaAhorro extends CuentaBancaria {
    // public void mostrarSaldo(){ ← ERROR
}
```

🔒 CLASE FINAL – NO SE HEREDA

```
final class Utilidades { /*...*/ }
// class X extends Utilidades{} ← ERROR
```

🔒 ATRIBUTO FINAL – CONSTANTE

```
class Config {
    final int PUERTO = 8080; // al declarar
```

INTERFACES

interface · implements · herencia múltiple de tipo

04

Concepto: contrato puro — define *qué* debe existir, no el cómo. Una clase puede implementar N interfaces a la vez.

DEFINICIÓN E IMPLEMENTACIÓN

```
interface Reproductor {
    // todos los métodos son public abstract
    void reproducir();
    void parar();
    void ajustarVolumen(int nivel);
}

class RepAudio implements Reproductor {
    @Override
    public void reproducir() {
        System.out.println("Play");
    }
    @Override
    public void parar() {
        System.out.println("Stop");
    }
    @Override
    public void ajustarVolumen(int n) {
        System.out.println("Vol: " + n);
    }
}
```

HERENCIA MÚLTIPLE DE TIPO

```
interface Nadador { void nadar(); }
interface Volador { void volar(); }

class Pato implements Nadador, Volador {
    @Override
    public void nadar() { System.out.println("Splash"); }
    @Override
    public void volar() { System.out.println("Fly"); }
}
```

ABSTRACTA + INTERFAZ COMBINADAS

```
abstract class Vehículo {
    public abstract void arrancar();
}

interface Electrico { void cargar(); }

// extends 1 clase + implements N interfaces
class CocheElectrico extends Vehículo
    implements Electrico {
    @Override public void arrancar() {
        System.out.println("Silencio");
    }
    @Override public void cargar() {
        System.out.println("Cargando");
    }
}
```

ABSTRACTA VS INTERFAZ

CARACTERÍSTICA	ABSTRACTA	INTERFAZ
Métodos con código	✓ Sí	✗ No (salvo default)

CLASES Y MÉTODOS GENÉRICOS

<T> · <K,V> · seguridad de tipos en compilación

EL PROBLEMA SIN GENÉRICOS

```
class Caja {                // ← usa Object
    private Object obj;
    public void set(Object o) { obj = o; }
    public Object get() { return obj; }
}
Caja c = new Caja();
c.set("Hola");
Integer n = (Integer) c.get(); // ClassCastException!
```

SOLUCIÓN CON GENÉRICOS – CAJA<T>

```
class Caja<T> {              // T = parámetro de tipo
    private T obj;
    public void set(T o) { obj = o; }
    public T get() { return obj; }
}

// Uso – String
Caja<String> cs = new Caja<>();
cs.set("Hola");
String s = cs.get(); // sin casting ✓
// cs.set(42); ← ERROR en compilación ✓

// Uso – Integer
Caja<Integer> ci = new Caja<>();
ci.set(42);
int n = ci.get(); // sin casting ✓
```

DOS PARÁMETROS DE TIPO – PAR<K,V>

```
class Par<K, V> {
    private K clave;
    private V valor;
    public Par(K c, V v) { clave=c; valor=v; }
    public K getClave() { return clave; }
    public V getValor() { return valor; }
}

Par<String,Integer> edad =
    new Par<>("Ana", 25);
System.out.println(edad.getClave()); // Ana
System.out.println(edad.getValor()); // 25
```

COLECCIONES – EL USO MÁS FRECUENTE

```
ArrayList<String> lista = new ArrayList<>();
lista.add("Java");
// lista.add(42); ← ERROR compilación
String x = lista.get(0); // sin casting

HashMap<String,Integer> m = new HashMap<>();
m.put("Ana", 25);
int e = m.get("Ana"); // sin casting
```

LETRA	CONVENCIÓN	EJEMPLO
T	Type (tipo genérico)	Caja<T>
E	Element (en listas)	List<E>
K	Key (clave)	Map<K,V>

05

EXCEPCIONES EN JAVA

try · catch · finally · throw · throws · custom

JERARQUÍA

```
Throwable |— Error ← NO capturar (JVM) | |— OutOfMemoryError | |—
StackOverflowError |— Exception ← si gestionar |— RuntimeException
← Unchecked ★ | |— ArithmeticException | |— NullPointerException |
|— ArrayIndexOutOfBoundsException | |— ClassCastException | |—
IllegalArgumentException | |— InputMismatchException |— IOException
← Checked ★ |— FileNotFoundException
```

TIPO	¿OBLIGA TRY-CATCH?	EJEMPLO
Checked	✓ Sí (compilador)	IOException
Unchecked	✗ No (recomendable)	RuntimeException
Error	Nunca capturar	OutOfMemoryError

TRY · CATCH · FINALLY

```
try {
    int r = 100 / divisor; // código peligroso
} catch (ArithmeticException e) {
    System.out.println(e.getMessage());
} catch (Exception e) { // más general al final
    System.out.println("Error genérico");
} finally {
    sc.close(); // SIEMPRE se ejecuta
}

// Multi-catch (Java 7+):
catch (IOException | SQLException e) { /*...*/ }
```

THROW VS THROWS

```
// throws → en la FIRMA: avisa que puede lanzar
public void leerArchivo() throws IOException {
    // throw → DENTRO: lanza en ese instante
    if (archivo == null)
        throw new IOException("No encontrado");
}

// El llamador DEBE capturar:
try { leerArchivo(); }
catch (IOException e) { /*...*/ }
```

	THROW	THROWS
Dónde	Cuerpo del método	Firma del método
Qué hace	Lanza la excepción	La declara/propaga

EXCEPCIÓN PERSONALIZADA

```
class SaldoInsuficienteException
    extends Exception {
    public SaldoInsuficienteException(String m) {
        super(m);
    }
}

// Uso en método:
public void retirar(double c)
    throws SaldoInsuficienteException {
    if (c ≤ 0)
        throw new IllegalArgumentException("Inválido");
    if (c > saldo)
```